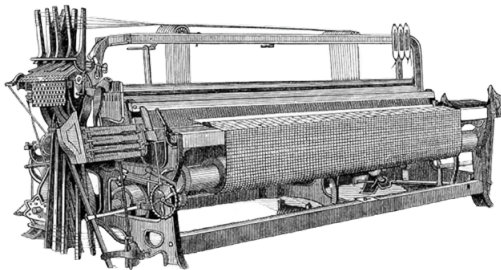


Multitarea en Sistemas Operativos de Propósito General

Técnicas Digitales III



“Any sufficiently advanced technology is indistinguishable from magic.”

Arthur C. Clarke

1 Repaso de Modo protegido

- Requerimientos de la arquitectura
- Registros disponibles
- Gestión de memoria y segmentación
- Paginación
- Cambio de contexto

2 Multiprocesamiento en GPOS

- ¿Que es un programa?
- Process & Threads
- Cambios de contexto
- User space vs Kernel space
- Política de Scheduling

Modos de Operación: *User & Kernel Space.*

Uno de los principales requerimientos fue la necesidad de separar el **user** y **kernel** space. Esto fue esencial para proteger el sistema operativo de acciones inadvertidas o malintencionadas por parte de los programas de usuario. Los sistemas operativos de propósito general como **UNIX** y sus derivados necesitaban que el procesador pudiera cambiar entre estos dos modos para proporcionar:

- **Protección de memoria:** Evitar que los programas de usuario modifiquen el código del sistema operativo.
- **Control de acceso a hardware:** Solo el kernel puede acceder directamente al hardware, lo que evita que los programas en modo usuario dañen o controlen dispositivos.

Soporte para Multitarea: *Cambios de Contexto.*

A medida que los sistemas operativos evolucionaron hacia la multitarea, los procesadores tuvieron que incorporar registros y mecanismos para realizar el cambio de contexto entre procesos. Esto significaba que los procesadores debían ser capaces de:

- **Guardar y restaurar el estado completo de un proceso.**
- **Soportar múltiples niveles de interrupciones y prioridad de tareas.**

Este requisito condujo al desarrollo de registros de propósito especial como el registro **CR3** (en x86) para la paginación y estructuras como la tabla de descriptores de segmento (**GDT/LDT**) para administrar la memoria de los procesos.

Gestión y Protección de Memoria: *Segmentación y Paginación.*

Los primeros sistemas operativos de propósito general, como **UNIX**, necesitaban soporte para la memoria virtual, lo que permitió a los programas utilizar más memoria de la que físicamente estaba disponible en la máquina. Esto requirió:

- **Segmentación:** Dividir el espacio de direcciones en segmentos lógicos como código, datos y pila.
- **Paginación:** Es necesario tener una gran capacidad de direccionamiento de memoria (concepto de **memoria virtual**).

Además, un GPOS requiere de una gestión protegida de la memoria en la que existan áreas **locales** y **globales**, siendo únicamente estas últimas aquellas accesibles para cualquier proceso.

Multitarea en Sistemas Operativos de Propósito General

Repaso de Modo protegido - Registros disponibles

General Purpose Registers

	31	16	15	0
EAX	AH Accumulator AL			
EBX	BH Base Index BL			
ECX	CH Count CL			
EDX	DH Data DL			
ESP	Stack Pointer			
EBP	Base Pointer			
EDI	Destination Index			
ESI	Source Index			

Control Registers

	31	16	15	0
CR3	Page Directory Base Address			
CR2				
CR1				
CR0				

Debug Registers

	31	16	15	0
DR7				
DR6				
DR5				
DR4				
DR3				
CR2				
DR1				
DR0				

Test Registers

	31	16	15	0
TR1				
TR0				

Segment Registers

	15	0
CS	Code	
DS	Data	
ES	Extra	
SS	Stack	
FS		
GS		

Program Registers

EIP	Instruction Pointer	
EFLAGS	FLAGS	

Memory Management Registers

	15	0	31	0	19	0
TR	TSS Selector		TSS Base Address		TSS Limit	
LDTR	LDTSS Selector		LDT Base Address		LDT Limit	
IDTR			IDT Base Address		IDT Limit	
GDTR			GDT Base Address		GDT Limit	

Segmentación:

En el modo protegido, la memoria está dividida en segmentos. Un segmento es un bloque contiguo de memoria con una longitud y propiedades específicas.

- Es un método para dividir la memoria en partes lógicas llamadas segmentos. Cada segmento tiene un segment base (la dirección de inicio) y un segment limit (el tamaño).
- El acceso a la memoria se realiza mediante direcciones lógicas, que consisten en dos componentes:
 - 1 Un **selector de segmento**: que indica qué segmento usar.
 - 2 Un **offset** dentro del segmento: que señala una posición específica dentro de ese segmento.

Cada vez que un programa accede a una dirección de memoria, el procesador usa esta información para localizar el segmento correspondiente y sumar el offset al segmento base para obtener la **dirección lineal**.

Descriptores y Tablas de Descriptores:

Los **segmentos** no se representan directamente en la memoria, sino a través de **descriptores** de segmento, que almacenan la información sobre cada segmento. Los descriptores incluyen:

- 1 La **base** del segmento.
- 2 El **límite** del segmento.
- 3 Permisos y atributos, como si el segmento es de código o de datos, y si es de solo lectura o escritura.

Estos descriptores están almacenados en **tablas de descriptores**, que son estructuras en la memoria que agrupan múltiples descriptores.

- **GDT**: Contiene descriptores que son comunes a todos los procesos.
- **LDT**: Contiene descriptores específicos para un proceso o grupo de procesos. Cada proceso puede tener su propia **LDT**.

Selectores de Segmento:

El **selector** es un valor de 16 bits que se utiliza en las instrucciones del procesador para referirse a un segmento. Un selector no contiene directamente la dirección del segmento, sino que apunta a un descriptor de segmento en la **GDT** o **LDT**. Los bits del selector se interpretan de la siguiente manera:

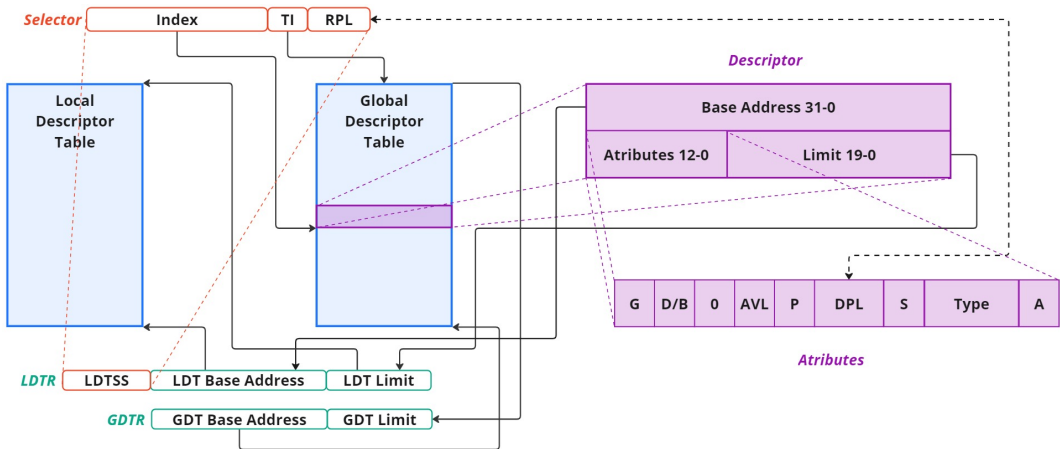
- Los bits 0-1 indican el **privilegio** del segmento (0 = kernel, 3 = usuario).
- El bit 2 indica si el segmento es parte de la **GDT** o la **LDT**.
- Los bits 3-15 son un índice en la tabla de descriptores.

Cuando el procesador recibe un selector como parte de una dirección lógica, utiliza los bits del selector para buscar el descriptor en la **GDT** o **LDT** y, a partir de ahí, calcula la **dirección lineal** usando la base del segmento y el offset.

Los selectores se deberán cargar en los registros de segmento.

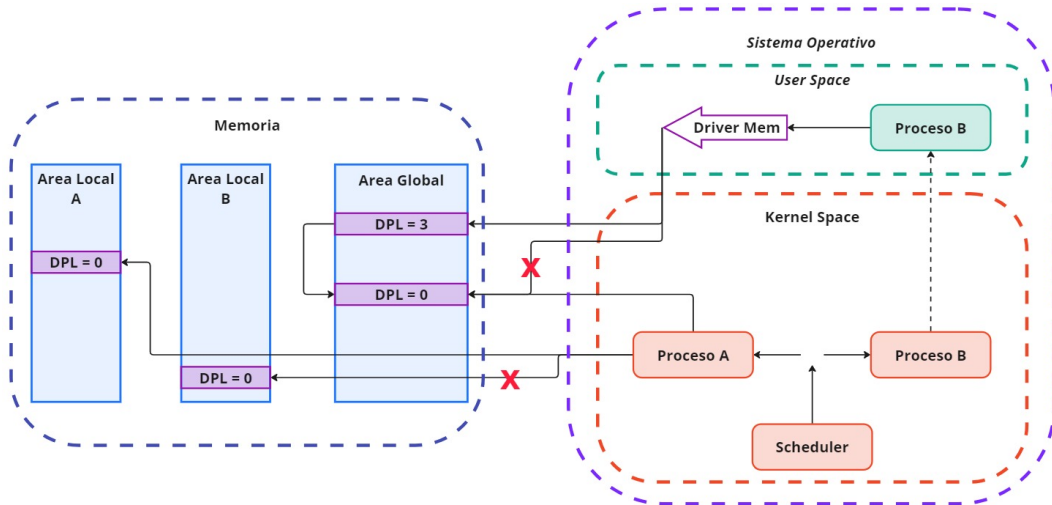
Multitarea en Sistemas Operativos de Propósito General

Repaso de Modo protegido - Gestión de memoria y segmentación



Multitarea en Sistemas Operativos de Propósito General

Repaso de Modo protegido - Gestión de memoria y segmentación



Paginación:

Una vez que el procesador ha convertido la dirección lógica en una dirección lineal (mediante segmentación), la paginación puede usarse para traducir esta dirección lineal a una dirección física en la memoria.

De esta manera se divide la memoria en bloques pequeños y de tamaño fijo llamados **páginas** (usualmente de 4kb en x86), en lugar de usar segmentos de tamaño variable, permitiendo que:

- La memoria física se gestione de manera eficiente.
- La memoria virtual pueda ser mucho mayor que la memoria física disponible, lo que permite al sistema usar **memoria virtual**.
- La memoria se organiza en estructuras de datos denominadas páginas, tablas de páginas y directorios de tablas de paginas.

Repaso de Modo protegido - Paginación

-
- El diagrama ilustra la arquitectura de memoria virtual. En la parte superior, se muestran los componentes hardware: la CPU, la memoria RAM y el disco duro. Debajo, se representa la estructura de memoria virtual, que incluye un Directorio, una Tabla de Páginas A, Páginas A1 y B1, una Tabla de Páginas B, y la Memoria Física. Las flechas indican el flujo de datos y la gestión de la memoria virtual.

El Proceso de Cambio de Contexto:

Durante el cambio de contexto entre dos procesos, el sistema operativo sigue varios pasos que involucran el **TSS**, la **GDT**, la **LDT**, y potencialmente la **IDT**. El proceso generalmente es activado por una interrupción del temporizador o una llamada al sistema que señala que es hora de ceder el CPU a otro proceso.

1 Interrupción o señal de cambio

- Una interrupción del temporizador o una llamada al sistema desde un proceso solicita al sistema operativo cambiar a otro proceso.
- El procesador accede a la **IDT** para encontrar la rutina de interrupción que maneja el cambio de contexto.

2 Guardar el estado actual (proceso saliente)

- Todos los registros de la CPU (puntero de pila, contador de programa, etc.).
- Información del segmento de código y datos
- Dirección de retorno de la interrupción (para continuar el proceso saliente en el futuro).

3 Acceso a la GDT/LDT

- El sistema accede a la **GDT** para encontrar el descriptor de **TSS** del proceso entrante.
- Si el proceso entrante tiene su propia **LDT**, el kernel ajusta el registro **LDTR** para que apunte a la nueva LDT, proporcionando acceso a los descriptors de segmento específicos de ese proceso.

4 Cargar el nuevo estado (proceso entrante)

- El sistema carga los valores de la **TSS del proceso entrante** en los registros de la CPU.
- Esto incluye el puntero de pila, el contador de programa, los registros generales, y cualquier otro dato relevante para restaurar el contexto de ejecución del proceso.

5 Reanudar la ejecución

- Una vez que el contexto del proceso entrante está cargado, el sistema operativo cede el control del CPU a ese proceso.
- El procesador empieza a ejecutar el código del proceso entrante desde el punto donde había quedado.

¿Qué rol juega el TSS en Linux Actualmente?

En **Linux**, el contexto de ejecución de un proceso no se guarda en el **Task State Segment** (TSS) como en los sistemas x86 tradicionales. Linux utiliza su propia estructura para gestionar la información de los procesos, llamada **Process Control Block** (PCB), que se implementa a través de la estructura llamada ***task struct***.

- El **TSS** se utiliza para manejar transiciones específicas como cambios de privilegio, por ejemplo, de modo de usuario a modo kernel.
- También se usa para gestionar interrupciones que requieren un cambio de nivel de privilegio, como cuando ocurre una interrupción y el sistema necesita cambiar a la **pila del kernel**.
- El TSS en **Linux** generalmente contiene solo el puntero de pila para la pila del kernel cuando ocurre una interrupción desde modo usuario, pero no se utiliza para almacenar el contexto completo del proceso.

Process Control Block (PCB) en Linux

El **PCB** en Linux, representado por la estructura ***task struct***, es una estructura en memoria que contiene toda la información relevante sobre un proceso, como:

- 1 Registros de CPU
- 2 Estado del proceso: En ejecución, bloqueado, en espera, etc.
- 3 Prioridades y políticas de planificación.
- 4 Identificadores: PID (Process ID), UID (User ID).
- 5 Dirección de memoria: Información sobre la memoria virtual asignada al proceso (paginación, tablas de páginas, etc.).
- 6 Recursos: Archivos abiertos, dispositivos, etc.

Cuando un cambio de contexto ocurre en Linux, el contexto de ejecución del proceso actual se guarda en su correspondiente ***task struct***, y luego el contexto del proceso entrante se restaura desde su ***task struct***.

¿Que es un programa?

Un programa, en términos generales, es un conjunto de instrucciones escritas en un lenguaje de programación que puede ser ejecutado por una computadora para realizar una tarea específica. Este conjunto de instrucciones está almacenado en un archivo y define una secuencia de operaciones a realizar.

No está en ejecución, solo existe en el almacenamiento como un archivo.

Cuando un programa se ejecuta, el kernel de Linux juega un papel fundamental en cargarlo en memoria y gestionar su interacción con el hardware y otros procesos.

Process

- Un proceso es una instancia de un programa en ejecución. Tiene su propio **espacio de direcciones de memoria**, lo que incluye su propia tabla de páginas, código, datos y pila. Los procesos son aislados entre sí, lo que garantiza que un proceso no puede acceder directamente a la memoria de otro.

Threads

- Un hilo es la **unidad básica** de ejecución dentro de un proceso. Comparte el mismo espacio de direcciones de memoria que los otros hilos de su proceso.
- Mientras los procesos tienen su propio contexto de ejecución, los hilos dentro del mismo proceso comparten muchos recursos (Ej. espacio de memoria), pero cada hilo tiene su propio conjunto de registros.

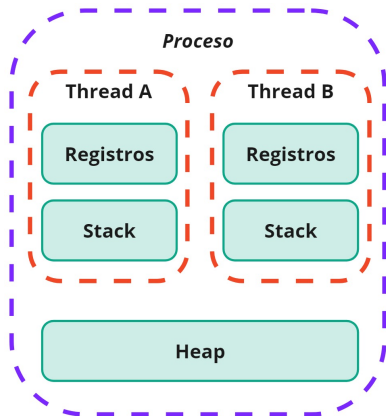
Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - Process & Threads

Process

- Cada proceso tiene su propio conjunto de **recursos del sistema**: su contexto de ejecución, incluyendo registros, espacio de direcciones virtuales, archivos abiertos, y más.

Threads



Cambios de contexto:

Process

- El sistema operativo debe guardar y cargar todo el contexto del proceso que incluye:
 - ➊ Registros de propósito general (EAX, EBX, etc.).
 - ➋ Registros de segmento (CS, DS, ES, SS), que deben ser actualizados consultando la **GDT** o **LDT**.
 - ➌ El registro **CR3**, que apunta a la tabla de páginas del proceso, cambiando así el espacio de direcciones virtuales.

Threads

- Como los hilos comparten el mismo espacio de **direcciones de memoria** (y por lo tanto la misma tabla de páginas), el sistema operativo no necesita cambiar el registro **CR3** ni recargar los descriptores de la **GDT** o **LDT**.

Cambios de contexto:

Process

- Al cambiar de un proceso a otro, puede ser necesario cargar una nueva **LDT** para ese proceso a través del registro **LDTR**, y actualizar la **GDT** si es necesario.

Threads

- Solo es necesario actualizar los registros específicos del hilo, como:
 - 1 El contador de programa (EIP), que apunta a la instrucción que ejecutará el hilo.
 - 2 El puntero de pila (ESP), porque cada hilo tiene su propia pila.
 - 3 Los registros de propósito general.
- El cambio de contexto entre hilos es **más rápido** que entre procesos, ya que no es necesario cambiar el espacio de memoria.

En User Space:

Process

- Procesos de usuario son aquellos que se ejecutan en **modo usuario**, lo que significa que tienen acceso restringido a los recursos del sistema. No pueden realizar operaciones privilegiadas como acceder directamente al hardware o modificar la memoria del kernel.

Threads

- Los threads a nivel de usuario son hilos gestionados completamente por una biblioteca de hilos en el espacio de usuario, **sin intervención directa del kernel**. Estos hilos solo pueden ejecutar código en modo usuario, y el kernel no es consciente de su existencia.

Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - User space vs Kernel space

En Kernel Space:

Process

- Procesos del kernel tienen acceso completo a los recursos del sistema, incluyendo el hardware y la memoria. El kernel ejecuta el código más privilegiado del sistema.

Threads

- Los threads a nivel de kernel son aquellos gestionados directamente por el kernel. Estos hilos pueden ejecutar código tanto en modo usuario como en modo kernel, dependiendo de las necesidades del sistema.

Los sistemas operativos modernos, como Linux y Windows, suelen implementar los hilos de usuario mediante hilos del kernel, lo que proporciona una mayor flexibilidad y rendimiento.

En User Space, sobre Paginación:

Process

- Los procesos a nivel de usuario tienen su propio espacio de direcciones virtuales, separado de otros procesos. El kernel utiliza la paginación para asignarles sus propias tablas de páginas.

Threads

- Los hilos a nivel de usuario dentro de un proceso comparten el mismo espacio de direcciones, por lo que no necesitan un cambio de tabla de páginas al cambiar entre hilos (solo se cambian los registros del hilo, como el contador de programa y la pila).

Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - User space vs Kernel space

En Kernel Space, sobre Paginación:

El kernel tiene su propio espacio de direcciones, que está mapeado por una parte del sistema de paginación, independiente del espacio de direcciones de los procesos de usuario. Cuando se realiza una llamada al sistema o una interrupción, el procesador cambia al espacio de direcciones del kernel.

Process

- Al cambiar de un proceso de usuario a código en modo kernel (por ejemplo, durante una llamada al sistema), no necesariamente se cambia todo el espacio de direcciones, sino que simplemente se accede a las páginas de memoria que el kernel ha mapeado.

Threads

- Los hilos a nivel de kernel operan con acceso completo al espacio de direcciones del kernel, y el cambio de contexto entre hilos del kernel puede implicar la actualización del registro **CR3** si los hilos están en diferentes procesos.

En User Space, sobre Cambios de Contexto:

Process

- En un cambio de contexto entre procesos de usuario, el kernel guarda el estado completo del proceso actual (registros, contador de programa, tabla de páginas, etc.) y carga el estado del nuevo proceso.

Threads

- Para hilos a nivel de usuario gestionados por una biblioteca de hilos, el cambio de contexto ocurre enteramente en el espacio de usuario. El kernel no se involucra y no es consciente de los cambios entre hilos de usuario, ya que ve todos los hilos como parte de un solo proceso.

En Kernel Space, sobre Cambios de Contexto:

Process

- Para los procesos de kernel o hilos gestionados por el kernel, el cambio de contexto es más ligero que entre procesos de usuario, ya que:
 - 1 No siempre es necesario cambiar el espacio de direcciones (ya que el kernel tiene acceso global).
 - 2 Los hilos del kernel tienen su propio conjunto de registros y pilas, pero comparten acceso completo al espacio de direcciones del kernel, por lo que no es necesario cambiar de espacio de memoria.

Threads

- El cambio de contexto entre hilos del kernel implica menos cambios en comparación con los procesos de usuario, porque no se necesita cambiar el espacio de direcciones, solo los registros de propósito general, el puntero de pila y el contador de programa.

Ventajas de threads a nivel de usuario vs hilos a nivel de kernel:

User Mode

- El cambio de contexto es más rápido porque no requiere intervención del kernel.
- Más flexibles en términos de planificación y administración por parte de la biblioteca de hilos en espacio de usuario.

Kernel Mode

- El kernel puede programar hilos en diferentes núcleos de manera efectiva.
- Si un hilo de kernel se bloquea, el kernel puede continuar ejecutando otros hilos dentro del mismo proceso.

Desventajas de threads a nivel de usuario vs hilos a nivel de kernel:

User Mode

- Si un hilo a nivel de usuario se bloquea, todo el proceso se bloquea porque el kernel no sabe que hay múltiples hilos.
- Los hilos de usuario no pueden beneficiarse del paralelismo en procesadores multinúcleo, ya que el kernel solo ve un proceso.

Kernel Mode

- El cambio de contexto es más costoso porque el kernel está involucrado directamente.
- Puede haber más sobrecarga en la gestión de hilos.

Políticas de Scheduling en Linux

Linux utiliza varios algoritmos de planificación. Las políticas más comunes son:

1 Completamente Justo (CFS - Completely Fair Scheduler):

- Es el **scheduler por defecto** en Linux desde la versión 2.6.23.
- Su objetivo es ofrecer una distribución equitativa del tiempo de CPU entre todos los procesos, de acuerdo con sus prioridades.
- **CFS** asigna un **tiempo virtual** a cada proceso, que es una representación del tiempo que un proceso debería haber consumido si todos los procesos se ejecutaran a la misma velocidad. Los procesos que tienen menos tiempo ejecutado en relación con su peso (prioridad) obtienen más tiempo de CPU.

2 Planificación por prioridades:

- Linux también permite que algunos procesos tengan una mayor o menor prioridad, utilizando el campo de prioridad (**nice** value).
- Los procesos con menor valor **nice** tienen mayor prioridad, mientras que los procesos con mayor valor **nice** obtienen menos tiempo de CPU.

Políticas de Scheduling en Linux

3 Planificación en Tiempo Real:

- **SCHED_FIFO** (First In, First Out): Los procesos se ejecutan hasta que terminan o se bloquean, sin ser interrumpidos por otros procesos que no tengan mayor prioridad.
- **SCHED_RR** (Round Robin): Similar a FIFO, pero con un quantum de tiempo. Los procesos son interrumpidos después de un tiempo predefinido, y luego se colocan al final de la cola.

4 Otras políticas:

- **SCHED_IDLE**: Procesos que solo deben ejecutarse cuando no hay nada más que hacer.
- **SCHED_BATCH**: Procesos que no necesitan interactuar con el usuario y pueden ser ejecutados en segundo plano con menor prioridad.

CFS: Completely Fair Scheduler

- El **CFS** es el algoritmo principal y está diseñado para hacer que la ejecución de los procesos sea lo más justa posible. Cada proceso en ejecución tiene un peso que está determinado por su prioridad (*nice* value). El planificador sigue una estructura llamada **red-black tree** para mantener el seguimiento del tiempo que cada proceso ha sido ejecutado.
- El objetivo es minimizar el **tiempo de espera virtual** de cada proceso. El proceso que ha sido ejecutado menos tiempo virtual es seleccionado para ejecutarse a continuación. El cálculo de cuánto tiempo puede ejecutarse un proceso antes de que otro lo reemplace está determinado por el **tiempo de expiración de su quantum**.

CFS: Completely Fair Scheduler

A continuación se muestra un ejemplo simplificado del código fuente del kernel de Linux relacionado con la planificación de procesos. Este código se encuentra en el archivo ***kernel/sched/fair.c***, donde se maneja el CFS.

```
/*
 * This is the core of the CFS scheduler: it inserts or updates
 * the task into the rb-tree:
 */
static void
enqueue_entity(struct cfs_rq *cfs_rq,
               struct sched_entity *se,
               int flags)
{
    /*
     * Update the vruntime before adding to the tree.
     */
    update_curr(cfs_rq);
    if (flags & ENQUEUE_WAKEUP) {
        se->vruntime += cfs_rq->min_vruntime;
        if (se->vruntime < cfs_rq->min_vruntime)
            se->vruntime = cfs_rq->min_vruntime;
    }
}
```

```
/* Add the task to the run queue's red-black tree */
rb_insert_sched_entity(cfs_rq, se);

/*
 * Update the load tracking statistics.
 */
account_entity_enqueue(cfs_rq, se);

/* If this is the first entity to be enqueued, update the min_vruntime */
if (se == cfs_rq->curr)
    cfs_rq->min_vruntime = se->vruntime;
}
```

Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - Política de Scheduling

CFS: Completely Fair Scheduler

```
/*
 * This is the core of the CFS scheduler: it inserts or updates
 * the task into the rb-tree:
 */
static void
enqueue_entity(struct cfs_rq *cfs_rq,
               struct sched_entity *se,
               int flags)
{
    /*
     * Update the vruntime before adding to the tree.
     */
    update_curr(cfs_rq);
    if (flags & ENQUEUE_WAKEUP) {
        se->vruntime += cfs_rq->min_vruntime;
        if (se->vruntime < cfs_rq->min_vruntime)
            se->vruntime = cfs_rq->min_vruntime;
    }
}
```

```
/* Add the task to the run queue's red-black tree */
rb_insert_sched_entity(cfs_rq, se);

/*
 * Update the load tracking statistics.
 */
account_entity_enqueue(cfs_rq, se);

/* If this is the first entity to be enqueued, update the min_vruntime */
if (se == cfs_rq->curr)
    cfs_rq->min_vruntime = se->vruntime;
}
```

enqueue_entity: Se encarga de agregar un proceso a la cola de ejecución del **CFS**, que es una **red-black tree**. Antes de insertar el proceso, se actualiza su **vruntime**, que se utiliza para decidir cuándo debe ser ejecutado en comparación con otros procesos.

Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - Política de Scheduling

CFS: Completely Fair Scheduler

```
/*  
 * This is the core of the CFS scheduler: it inserts or updates  
 * the task into the rb-tree:  
 */  
static void  
enqueue_entity(struct cfs_rq *cfs_rq,  
               struct sched_entity *se,  
               int flags)  
{  
    /*  
     * Update the vruntime before adding to the tree.  
     */  
    update_curr(cfs_rq);  
    if (flags & ENQUEUE_WAKEUP) {  
        se->vruntime += cfs_rq->min_vruntime;  
        if (se->vruntime < cfs_rq->min_vruntime)  
            se->vruntime = cfs_rq->min_vruntime;  
    }  
}
```

```
/* Add the task to the run queue's red-black tree */  
rb_insert_sched_entity(cfs_rq, se);  
  
/*  
 * Update the load tracking statistics.  
 */  
account_entity_enqueue(cfs_rq, se);  
  
/* If this is the first entity to be enqueued, update the min_vruntime */  
if (se == cfs_rq->curr)  
    cfs_rq->min_vruntime = se->vruntime;  
}
```

pick_next_entity: Selecciona el siguiente proceso a ejecutar, buscando el nodo más a la izquierda en el árbol de procesos. Este proceso es el que tiene el **vruntime** más bajo, es decir, el que ha sido ejecutado menos tiempo en relación con su prioridad.

Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - Política de Scheduling

Context Switch

En **Linux**, el cambio de contexto se gestiona en la función ***context_switch()***, la cual es llamada por el planificador del sistema cuando determina que es necesario cambiar de proceso. Esta función está implementada en el archivo ***kernel/sched/core.c***.

```
/*
 * context_switch - Perform the actual process context switch.
 * @prev: the task being switched out.
 * @next: the task being switched in.
 *
 * This function switches the current running task from @prev to @next
 * by saving the context of @prev and restoring the context of @next.
 */
static inline void
context_switch(struct rq *rq,
               struct task_struct *prev,
               struct task_struct *next)
{
    struct mm_struct *mm, *oldmm;

    /* Switch the address space, if necessary */
    mm = next->mm;
    oldmm = prev->active_mm;
```

```
    if (mm != oldmm) {
        /* If the next process has its own memory context, switch to it */
        switch_mm(oldmm, mm, next);
    } else {
        /* Keep the memory context of the previous process */
        if (!next->mm)
            enter_lazy_tlb(oldmm, next);
    }

    /* Now we prepare to switch to the new process's kernel stack */
    rq->curr = next;

    /* Actually switch the kernel context (CPU registers, etc.) */
    switch_to(prev, next, prev);

    /* When we return from switch_to, we are running the "next" task */
}
```

Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - Política de Scheduling

Context Switch

```
/*
 * context_switch - Perform the actual process context switch.
 * @prev: the task being switched out.
 * @next: the task being switched in.
 *
 * This function switches the current running task from @prev to @next
 * by saving the context of @prev and restoring the context of @next.
 */
static inline void
context_switch(struct rq *rq,
               struct task_struct *prev,
               struct task_struct *next)
{
    struct mm_struct *mm, *oldmm;

    /* Switch the address space, if necessary */
    mm = next->mm;
    oldmm = prev->active_mm;
```

```
    if (mm != oldmm) {
        /* If the next process has its own memory context, switch to it */
        switch_mm(oldmm, mm, next);
    } else {
        /* Keep the memory context of the previous process */
        if (!next->mm)
            enter_lazy_tlb(oldmm, next);
    }

    /* Now we prepare to switch to the new process's kernel stack */
    rq->curr = next;

    /* Actually switch the kernel context (CPU registers, etc.) */
    switch_to(prev, next, prev);

    /* When we return from switch_to, we are running the "next" task */
}
```

context_switch(): Es la función principal responsable de cambiar entre dos procesos. Recibe como parámetros el proceso anterior y el nuevo.

Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - Política de Scheduling

Context Switch

```
/*
 * context_switch - Perform the actual process context switch.
 * @prev: the task being switched out.
 * @next: the task being switched in.
 *
 * This function switches the current running task from @prev to @next
 * by saving the context of @prev and restoring the context of @next.
 */
static inline void
context_switch(struct rq *rq,
               struct task_struct *prev,
               struct task_struct *next)
{
    struct mm_struct *mm, *oldmm;

    /* Switch the address space, if necessary */
    mm = next->mm;
    oldmm = prev->active_mm;
```

```
    if (mm != oldmm) {
        /* If the next process has its own memory context, switch to it */
        switch_mm(oldmm, mm, next);
    } else {
        /* Keep the memory context of the previous process */
        if (!next->mm)
            enter_lazy_tlb(oldmm, next);
    }

    /* Now we prepare to switch to the new process's kernel stack */
    rq->curr = next;

    /* Actually switch the kernel context (CPU registers, etc.) */
    switch_to(prev, next, prev);

    /* When we return from switch_to, we are running the "next" task */
}
```

switch_mm(oldmm, mm, next): Gestiona el cambio del contexto de memoria entre procesos. Si el nuevo proceso tiene un espacio de direcciones diferente, se realiza un cambio de las tablas de páginas.

Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - Política de Scheduling

Context Switch

```
/*
 * context_switch - Perform the actual process context switch.
 * @prev: the task being switched out.
 * @next: the task being switched in.
 *
 * This function switches the current running task from @prev to @next
 * by saving the context of @prev and restoring the context of @next.
 */
static inline void
context_switch(struct rq *rq,
               struct task_struct *prev,
               struct task_struct *next)
{
    struct mm_struct *mm, *oldmm;

    /* Switch the address space, if necessary */
    mm = next->mm;
    oldmm = prev->active_mm;
```

```
    if (mm != oldmm) {
        /* If the next process has its own memory context, switch to it */
        switch_mm(oldmm, mm, next);
    } else {
        /* Keep the memory context of the previous process */
        if (!next->mm)
            enter_lazy_tlb(oldmm, next);
    }

    /* Now we prepare to switch to the new process's kernel stack */
    rq->curr = next;

    /* Actually switch the kernel context (CPU registers, etc.) */
    switch_to(prev, next, prev);

    /* When we return from switch_to, we are running the "next" task */
}
```

enter_lazy_tlb(): Si el proceso entrante no tiene su propio espacio de direcciones (por ejemplo, un hilo que comparte el espacio de direcciones con otro proceso), se mantiene el espacio de direcciones anterior pero en un modo más eficiente.

Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - Política de Scheduling

Context Switch

```
/*
 * context_switch - Perform the actual process context switch.
 * @prev: the task being switched out.
 * @next: the task being switched in.
 *
 * This function switches the current running task from @prev to @next
 * by saving the context of @prev and restoring the context of @next.
 */
static inline void
context_switch(struct rq *rq,
               struct task_struct *prev,
               struct task_struct *next)
{
    struct mm_struct *mm, *oldmm;

    /* Switch the address space, if necessary */
    mm = next->mm;
    oldmm = prev->active_mm;
```

```
    if (mm != oldmm) {
        /* If the next process has its own memory context, switch to it */
        switch_mm(oldmm, mm, next);
    } else {
        /* Keep the memory context of the previous process */
        if (!next->mm)
            enter_lazy_tlb(oldmm, next);
    }

    /* Now we prepare to switch to the new process's kernel stack */
    rq->curr = next;

    /* Actually switch the kernel context (CPU registers, etc.) */
    switch_to(prev, next, prev);

    /* When we return from switch_to, we are running the "next" task */
}
```

switch_to(prev, next, prev): Esta macro escrita en assembler guarda el estado del proceso anterior y restaura el estado del proceso siguiente. Tras esta función, el nuevo proceso estará ejecutándose.

Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - Política de Scheduling

switch_to(prev, next, prev)

```
pushfl          # Save flags
pushl   %ebp    # Save base pointer
movl   %esp, prev # Save stack pointer (prev->thread.sp)
movl   next, %esp # Restore stack pointer of next (next->thread.sp)
movl   $1f, prev_ip # Save return address (prev->thread.ip)
pushl  next_task # Push next task address
jmp    __switch_to # Jump to __switch_to function
```

```
1:              # Label to return to
popl   %ebp     # Restore base pointer
popfl              # Restore flags
```

Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - Política de Scheduling

Thread Schedule

La creación de un nuevo hilo (o proceso) en Linux se realiza a través de la llamada al sistema ***clone()***, que es similar a ***fork()***, pero con más control sobre qué recursos se comparten entre el proceso padre e hijo (en el caso de hilos, comparten el espacio de direcciones). Fragmento extraído de ***kernel/fork.c***.

```
long do_fork(unsigned long clone_flags, unsigned long stack_start, unsigned long stack_size, int __user *parent_tidptr, int __user *child_tidptr)
{
    struct task_struct *p;
    int trace = 0;
    long nr;

    /* Allocate task_struct for the new process/thread */
    p = copy_process(clone_flags, stack_start, stack_size, child_tidptr, NULL);
    if (!IS_ERR(p)) {
        /* Add the new thread to the scheduler */
        wake_up_new_task(p);

        /* Assign PID to the new task (thread) */
        nr = task_pid_vnr(p);

        /* Return the new process ID to the parent process */
        return nr;
    }
    return PTR_ERR(p);
}
```

Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - Política de Scheduling

Thread Schedule

```
long do_fork(unsigned long clone_flags, unsigned long stack_start, unsigned long stack_size, int __user *parent_tidptr, int __user *child_tidptr)
{
    struct task_struct *p;
    int trace = 0;
    long nr;

    /* Allocate task_struct for the new process/thread */
    p = copy_process(clone_flags, stack_start, stack_size, child_tidptr, NULL);
    if (!IS_ERR(p)) {
        /* Add the new thread to the scheduler */
        wake_up_new_task(p);

        /* Assign PID to the new task (thread) */
        nr = task_pid_vnr(p);

        /* Return the new process ID to the parent process */
        return nr;
    }
    return PTR_ERR(p);
}
```

do_fork(): Esta es la función central que maneja la creación de un nuevo proceso o hilo. Si el flag **CLONE_VM** está activado, el nuevo hilo comparte el mismo espacio de direcciones que el proceso padre (esto es lo que crea un hilo en lugar de un proceso separado).

Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - Política de Scheduling

Thread Schedule

```
long do_fork(unsigned long clone_flags, unsigned long stack_start, unsigned long stack_size, int __user *parent_tidptr, int __user *child_tidptr)
{
    struct task_struct *p;
    int trace = 0;
    long nr;

    /* Allocate task_struct for the new process/thread */
    p = copy_process(clone_flags, stack_start, stack_size, child_tidptr, NULL);
    if (!IS_ERR(p)) {
        /* Add the new thread to the scheduler */
        wake_up_new_task(p);

        /* Assign PID to the new task (thread) */
        nr = task_pid_vnr(p);

        /* Return the new process ID to the parent process */
        return nr;
    }
    return PTR_ERR(p);
}
```

copy_process(): Aquí se copia la estructura del proceso actual (***task_struct***) y se configura el nuevo hilo con los flags especificados por el usuario. Esto determina qué recursos comparte el nuevo hilo con el proceso padre.

Multitarea en Sistemas Operativos de Propósito General

Multiprocesamiento en GPOS - Política de Scheduling

Thread Schedule

```
long do_fork(unsigned long clone_flags, unsigned long stack_start, unsigned long stack_size, int __user *parent_tidptr, int __user *child_tidptr)
{
    struct task_struct *p;
    int trace = 0;
    long nr;

    /* Allocate task_struct for the new process/thread */
    p = copy_process(clone_flags, stack_start, stack_size, child_tidptr, NULL);
    if (!IS_ERR(p)) {
        /* Add the new thread to the scheduler */
        wake_up_new_task(p);

        /* Assign PID to the new task (thread) */
        nr = task_pid_vnr(p);

        /* Return the new process ID to the parent process */
        return nr;
    }
    return PTR_ERR(p);
}
```

wake_up_new_task(): Una vez que el hilo está creado, esta función lo agrega a la queue de tareas del scheduler, permitiendo que sea programado para ejecutarse.